

Cross-Dataset CICIDS2017 ↔ CSE-CIC-IDS2018 — สารบัญผลการทดลอง

รวมผลทุกรอบของงาน cross-dataset แยกตามรอบที่ทำ เพื่อให้ย้อนดูและเปรียบเทียบได้

แต่ละรอบแยกเป็นสามส่วนเหมือนกันหมด

```
roundN_ /
├── report/                .md                .pdf
├── data/
└── code/                  (                  )
```

ทุกตัวเลขในโฟลเดอร์นี้มาจากไฟล์ผลลัพธ์จริง แต่ละรอบเก็บทั้งรายงาน ข้อมูลดิบที่ใช้สร้าง ตาราง และสคริปต์ที่รัน จึงตรวจย้อนกลับได้ทุกตัวเลข

คำเตือนก่อนเปรียบเทียบข้ามรอบ

ตัวเลขข้ามรอบเทียบกันตรง ๆ ไม่ได้ แต่ละรอบเปลี่ยนเงื่อนไขไปหลายอย่างพร้อมกัน ตารางด้านล่าง ระบุว่ารอบไหนเปลี่ยนอะไร ให้ดูคอลัมน์เงื่อนไขก่อนเสมอแล้วค่อยดูตัวเลข

	รอบ 2	รอบ 3	รอบ 4	รอบ 5
ชุดที่ใช้เทรน	2018	2017	ทั้งสองฝั่ง	ทั้งสองฝั่ง
คลาสตอนเทรน	15 และ 7	9 แล้วยุบเป็น 7	7 ตั้งแต่ต้น	7 ตั้งแต่ต้น
Features	76 และ 77	77	77	77
การ clean	ไม่ได้ทำ	ไม่ได้ทำ	กฎเดียวกันสองฝั่ง	clean ก่อน sample ทั้งสองฝั่ง
การแบ่งข้อมูล	70/30 ตามเวลา	70/30 ตามเวลา	60/10/30 สองแบบ	60/10/30 สองแบบ
Seeds	1	1	5	5
โมเดล	4	7	7	7

รอบ 1 — สรุปข้อเสนอแนะและ mapping

□ round1_advisor_plan/

คำถาม ต้องทำอะไรบ้าง และสอง dataset เชื่อมกันได้อย่างไร

ผลลัพธ์ mapping 77 shared features ระหว่างสอง dataset โดยจับคู่จากลำดับคอลัมน์ ไม่ใช่จาก การเดาชื่อ พบว่า 27 คอลัมน์ชื่อตรงกันอยู่แล้ว 50 คอลัมน์ต้องแปลง ในนั้นมี 12 คอลัมน์ที่สลับ ลำดับค่า และมีกับดักหนึ่งจุดคือ CWE Flag Count ที่ฝั่ง 2018 ลืมย่อ Count เป็น Cnt

ยืนยันภายหลัง mapping นี้ตรงกับ feature_columns.json และไฟล์ข้อมูลจริงของฝั่ง 2017 ทั้ง 77 ตัว เรียงตรงกันทุกตำแหน่ง

ไฟล์	เนื้อหา
report/advisor_cross_dataset_summary.pdf	สรุปข้อเสนอแนะ สถานะงาน และแผน
data/shared_features_77.json	ตารางแปลงชื่อคอลัมน์ ใช้ในโค้ดได้ทันที

รอบ 2 — ราคาของการย้ายไป shared schema (ภายในชุด 2018)

□ round2_schema_ablation_2018/

คำถาม การเปลี่ยนไปใช้ schema ร่วมทำให้ผลเปลี่ยนไปเท่าไร และเปลี่ยนเพราะอะไร

วิธี สีเงื่อนไข เปลี่ยนทีละอย่าง บนตัวอย่าง 300k ของ 2018 split เดิม seed เดิม

เงื่อนไข	feature	คลาส
A ฐานเดิม	77 (มี Protocol)	15
B shared	76	15
B+ shared + Dst Port	77	15
C shared + ยุบคลาส	76	7

ผลสำคัญ การยุบ 15 คลาสเหลือ 7 ทำให้ตัวเลขพาดหัวดีขึ้นแต่ประสิทธิภาพจริงแย่ง

lightgbm	macro ทุกคลาส	macro เฉพาะคลาสที่วัดได้
ก่อนยุบ	0.7126	0.8575
หลังยุบ	0.7880	0.8193

ตัวเลขสองชุดขึ้นคนละทิศ เพราะการยุบคลาสเอาคลาสที่ทำได้ดีไปรวมกับคลาสที่พัง และซ่อนคลาสจิวที่เคยได้ศูนย์ออกจากการนับ

ผลอื่น: ดัด Protocol ไม่มีราคาต้องจ่าย (สามในสี่โมเดลขยับไม่ถึง 0.002) และ Destination Port ช่วย Infiltration จาก 0.1185 เป็น 0.1703

รอบ 3 — Cross-dataset ครั้งแรก ด้วยโมเดลที่มีอยู่เดิม

□ round3_transfer_handoff_models/

คำถาม ทำ cross-dataset ให้รันได้ก่อน ยังไม่เน้นผลดี

วิธี ใช้โมเดลที่เทรนไว้แล้วจากบันเดิล cicids2017_temporal_v1 (เทรนด้วย 9 คลาส) ทำนาย ข้อมูล 2018 แล้วยุบผลเป็น 7 คลาส ไม่ต้องใช้ข้อมูลดิบ 2017

ผลสำคัญ เทียบกับการเดาว่าทุกแถวเป็น Benign (macro-F1 0.1296)

โมเดล	macro-F1	กู้คืนได้
stacking	0.2428	20.3%
lightgbm	0.2298	16.2%
random_forest	0.1348	0.8%

random_forest ได้เท่าการเดาพอดี ส่วน mlp กับ logistic regression ได้ accuracy ต่ำกว่าการเดา

ตอนนั้นสรุปว่าสิ่งเดียวที่ถ่ายทอดได้คือความรู้เรื่อง traffic ปกติ (Benign 0.87–0.93) ส่วนคลาสโจมตีพังเกือบหมด ข้อสรุปนี้กว้างเกินไป รอบ 4 พบว่าในทิศ 2018 ไป 2017 ซึ่งรอบ 3 วัดไม่ได้ นั้น Bot ทำได้ 0.94 และ Brute Force ทำได้ 0.91 ส่วนในทิศที่รอบ 3 วัด ข้อสรุปเดิมยังคงถูกต้อง

การทดลองแทรก บังคับให้โมเดลตอบได้แค่ 7 คลาสรวม (ดัด probability ของ Heartbleed และ PortScan ก่อน argmax) ดีขึ้นสูงสุด 0.008 และมีสองโมเดลแย่ง จึงประเมินตอนนั้นว่าการทำคลาสให้ เท่ากันมีผลราว 1% — การประเมินนี้วัดได้เพียงองค์ประกอบเดียว และยังไม่มีการวัดผลรวมของการทำคลาสให้เท่ากันแบบแยกตัวแปรได้ ดูหัวข้อเทียบรอบ 3 กับรอบ 4

รอบ 4 — Protocol เดิมรูปแบบ

☐ round4_protocol_full/ * ผลดิบอยู่ที่ results/crossdataset/protocol_v1/

คำถาม ทำตาม protocol ให้ครบทุกข้อ แล้ววัดทั้งสี่ทิศทาง

วิธี 77 features * 7 คลาสเทรנדั้งแต่ต้น * 300k ต่อ dataset * 60/10/30 * 5 seeds * สองวิธีแบ่ง * 7 โมเดล * ไม่ทวน → 280 แถวผล

ผลที่หนึ่ง — การแบ่งแบบสุ่มทำให้เพดานเพ้อ และเพ้อไม่เท่ากัน

ชุด	เพ้อขึ้นเท่าไร
2017	0.21 ถึง 0.32
2018	0.006 ถึง 0.033

ต่างกันสิบเท่า 2017 ภายใต้การแบ่งแบบสุ่มได้ 0.96–0.98 ซึ่งควรตั้งข้อสงสัยทันที

ผลที่สอง — การถ่ายทอดไม่สมมาตร และวิธีแบ่งข้อมูลปิดบังข้อนี้ไว้

วิธีแบ่ง	2017→2018	2018→2017
สุ่ม (ตาม protocol)	31.1%	25.8%
ตามลำดับเวลา	7.5%	73.9%

ถ้ารู้เฉพาะการแบ่งแบบสุ่มจะสรุปว่าสองทิศยากพอๆกัน ซึ่งไม่ตรงกับความจริง วิธีแบ่งข้อมูล เปลี่ยนข้อสรุป ไม่ใช่แค่เปลี่ยนตัวเลข

ผลที่สาม — ห้าในเจ็ดคลาสถ่ายทอดได้จริง

stacking เทรนด้วย 2018 ทดสอบกับ 2017 แบบแบ่งตามลำดับเวลา

คลาส	F1
Benign	0.951
Bot	0.936
Brute Force	0.883
DoS	0.640
DDoS	0.620
รวม 5 คลาส	0.806
Infiltration	0.000
Web Attack	0.000

รอบ 5 — ตรวจสอบความน่าเชื่อถือของผล ☐ รอบล่าสุด

☐ round5_reliability_audit/ * รายงาน report/advisor_checklist_20260928.md (และ .pdf)

คำถาม ผลของรอบ 4 เชื่อได้แค่ไหน ตอบรายการตรวจสอบสี่ข้อของอาจารย์

สิ่งที่เปลี่ยนก่อนตรวจ สร้างคลัง 2018 ใหม่จากไฟล์ดิบ 10 ไฟล์ แล้ว clean ก่อน sample เหมือน ฝั่ง 2017 ทั้งสองฝั่งได้ครบ 300,000 แถวพอดี ผลดิบอยู่ที่ results/crossdataset/protocol_v2/

ข้อ	คำถาม	คำตอบ
1	AUC 1.0 ของ adversarial validation เป็น artifact ไหม	ไม่ใช่ label-shuffle ได้ 0.49–0.51 ไม่มีแถวข้ามคร่อม split ใน 30 การสุ่ม LR ก็แยกได้ 0.9478 ขึ้นไป
2	คนละ file/time ภายใน dataset แยกได้ง่ายไหม	แยกได้ แต่ไม่ง่ายเท่าข้าม dataset 0.66–0.94 เทียบ 0.95–1.00 ทุกโมเดล และโปรแกรมโจมตีตัวเดียวกันข้าม dataset ยังได้ 0.9999–1.0000
3	feature ใดเป็น fingerprint อย่างเดียว	แทบไม่มี มีตัวเดียวคือ act_data_pkt_fwd คอลัมน์ที่นอก dataset กับคอลัมน์ที่จำแนก attack เป็นชุดเดียวกัน (rank correlation +0.72 ถึง +0.77)
4	asymmetry และผลของ split mode ยังอยู่ไหม	ยังอยู่ทั้งคู่ asymmetry 7 จาก 7 โมเดล เล็กลงจาก 40.4 เป็น 33.6 จุด การแบ่งแบบสุ่มยังช้อนมันไว้ ผลดีที่สุดเดิม stacking 73.9% เหลือ 40.2%

คะแนนจริงควบคู่ Recovery % (3 ตุลาคม 2569)

□ round5_reliability_audit/report/macro_f1_with_recovery_20261003.md (และ .pdf) · ตารางต้นทาง results/crossdataset/protocol_v2/summary_macro_f1.csv (ไม่ได้รับการทดลองใหม่)

macro-F1 เฉลี่ย $\pm$ SD ของ 5 seeds โหมด chronological protocol_v2 (baseline เดว่าทุกแถวเป็น Benign = 0.1337 บน 2018 และ 0.1294 บน 2017) ในวงเล็บคือ Recovery % ตัวเลขเดิม

โมเดล	2017 ไป 2018: macro-F1 $\pm$ SD (Recovery)	2018 ไป 2017: macro-F1 $\pm$ SD (Recovery)
lightgbm	0.2058 $\pm$ 0.0641 (9.9%)	0.4480 $\pm$ 0.0079 (52.8%)
stacking	0.1715 $\pm$ 0.0007 (5.4%)	0.3674 $\pm$ 0.0386 (40.2%)
xgboost	0.1712 $\pm$ 0.0001 (5.2%)	0.3615 $\pm$ 0.0182 (39.5%)
logistic_regression	0.1508 $\pm$ 0.0000 (2.8%)	0.3493 $\pm$ 0.0000 (79.8%)
mlp	0.1755 $\pm$ 0.0014 (6.1%)	0.3107 $\pm$ 0.0248 (39.7%)
catboost	0.1726 $\pm$ 0.0009 (5.4%)	0.2226 $\pm$ 0.0041 (15.9%)
random_forest	0.1717 $\pm$ 0.0001 (5.4%)	0.1680 $\pm$ 0.0007 (7.2%)

- อันดับตาม Recovery กับตามคะแนนจริงไม่ตรงกัน logistic_regression ทิศ 2018 ไป 2017 ได้ Recovery สูงสุด (79.8%) แต่ macro-F1 จริงเป็นอันดับ 4 เพราะ ceiling ของมันบน 2017 ต่ำ (0.4050) lightgbm ได้คะแนนจริงสูงสุดและชนะทุก seed
- lightgbm ทิศ 2017 ไป 2018 ถูกดึงขึ้นโดย seed เดียว seed 42 ได้ 0.3204 อีก 4 seeds ได้ 0.1766 ถึง 0.1776
- SD ในโหมด chronological คือความสุ่มของโมเดลอย่างเดียว การแบ่งและตัวอย่าง 300,000 แถวไม่เปลี่ยนแปลงตาม seed โมเดลที่ไม่มีส่วนสุ่ม (logistic_regression) จึงได้ SD = 0.0000 พอดี
- ceiling ของ 2017 แบบ chronological ถูกกดโดย Brute Force ($F1 \approx 0$ ในทุกโมเดล เพราะ train ไม่มี SSH-Patator) ทำให้ Recovery ของทิศ 2018 ไป 2017 สูงขึ้น ดู
../ids2017/report/chronological_split_explained.md

เทียบรอบ 3 กับรอบ 4 ให้ถูกต้อง

รอบ 3 วัดได้ทิศเดียวคือ 2017 ไป 2018 ส่วนรอบ 4 วัดได้ทั้งสองทิศ การเทียบต้องดูทิศเดียวกัน เท่านั้น

ทิศ 2017 ไป 2018 ซึ่งเป็นทิศเดียวที่ทั้งสองรอบวัด — รอบ 4 ได้ต่ำกว่า

โมเดล	รอบ 3 Brute Force	รอบ 4 Brute Force
stacking	0.7359	0.0000
xgboost	0.3902	0.0000
lightgbm	0.2654	0.0000
logistic_regression	0.2394	0.0924

แต่สรุปไม่ได้ว่ารอบ 4 แยกว่า เพราะเงื่อนไขต่างกันหลายอย่างพร้อมกัน

- รอบ 3 โมเดลเทรนด้วย 2017 ทั้ง corpus 1,748,579 แถว ส่วนรอบ 4 เทรนด้วย 180,000 แถว
- รอบ 3 ไม่ได้ clean ผัง 2018 ส่วนรอบ 4 ตัดแถวซ้ำออก ทำให้ Brute Force ของ 2018 เหลือ 1,798 จาก 7,039 แถว
- วิธีแบ่งข้อมูลต่างกัน

ตัวเลข 0.88 ถึง 0.91 เป็นของทิศ 2018 ไป 2017 ซึ่งรอบ 3 วัดไม่ได้

โมเดล	Brute Force ทิศ 2018 ไป 2017
mlp	0.9145
xgboost	0.9017
lightgbm	0.8888
stacking	0.8831

นี่คือของใหม่จริงของรอบ 4 ไม่ใช่การปรับปรุงตัวเลขเดิม

เรื่องผลของการทำให้ชุดคลาสเท่ากัน — ยังไม่มีการวัดที่สะอาด

รอบ 3 ประเมินไว้ราว 1% จากการบังคับให้โมเดลตอบได้แค่ 7 คลาส ซึ่งวัดได้เพียงองค์ประกอบเดียว คือคำทำนายที่เสียไปกับคลาสที่ไม่มีในเป้าหมาย

รอบ 4 เปลี่ยนหลายอย่างพร้อมกัน จึงแยกไม่ได้ว่าส่วนไหนมาจากการทำคลาสให้เท่ากัน ทั้งสองรอบ จึงยังไม่ได้ตอบคำถามนี้ ถ้าต้องการคำตอบต้องรันเงื่อนไขที่คุมทุกอย่างให้เหมือนกันแล้วเปลี่ยน เฉพาะชุดคลาส

สิ่งที่รอบ 4 เพิ่มขึ้นจริง

ข้อ	รอบ 3	รอบ 4
ทิศทางที่วัดได้	1	4 combination
Seeds	1	5 พร้อมค่าเบี่ยงเบน
วิธีแบ่งข้อมูล	1	2 และพบว่ามันเปลี่ยนข้อสรุป
การ clean	ไม่ได้ทำ	กฎเดียวกันสองฝั่ง
ที่มาของโมเดล	ของผู้อื่น ยืนยันไม่ได้	เทรนเองทั้งหมด

สองคลาสที่ไม่ถ่ายทอดในทุกรอบ

Infiltration และ Web Attack ได้ F1 เท่ากับศูนย์ในทุกรอบ ทุกทิศ ทุกโมเดล ทุก seed ด้วย เหตุผลคนละแบบ

คลาส	2017	2018	สาเหตุ
Web Attack	2,143	17 (รอบ 5: 859)	ข้อมูลไม่พอ ฝั่งหนึ่งแทบไม่มีเลย รอบ 5 แก้เรื่องจำนวนแล้ว
Infiltration	36	2,946	ข้อมูลไม่พอ และ label เสีย

Infiltration น้อยกว่า เพราะแม้ทิศที่มีข้อมูลเทรน 1,700 แฉาก็ยังได้ศูนย์ และแม้แต่ตอนเทรนกับ
ทดสอบภายในชุดเดียวกันยังได้เพียง 0.05–0.13 ในทุกโมเดล

ข้อจำกัดที่ยังเหลือ

1. [แก้แล้วในรอบ 5] ฝั่ง 2018 หลัง clean เคยเหลือ 262,701 แฉา ไม่ครบ 300,000 ตาม protocol
ตอนนี้สร้างคลังใหม่จากไฟล์ดิบ ได้ครบ 300,000 แฉา
2. ช่อง calibration 10% กันไว้แล้วแต่ยังไม่ได้ใช้
3. ยังไม่ได้หุน hyperparameter
4. [ทำแล้วในรอบ 5] adversarial validation พร้อม control สามชั้น
5. ตัวบ่งชี้อัตราต่างเวลาต่างกัน 2017 ใช้ลำดับแฉาในไฟล์ 2018 ใช้ timestamp จริง ลำดับแฉาของ 2017
ตรงกับตารางเวลาการโจมตีระดับ ก่อนการโจมตี แต่ไม่ใช่ในภาพ และมีการที่ขัดกับตารางเวลา ดู
../ids2017/report/chronological_split_explained.md
6. ceiling ของ 2017 แบบ chronological ถูกกดโดย Brute Force train ของการแบ่ง 60/10/30 ไม่มี
SSH-Patator เลย (อยู่ในช่อง calibration ที่ไม่ได้ใช้) และ test เป็น SSH-Patator ล้วน Brute Force จึงได้
F1 ≈ 0 ในทุกโมเดล ส่วนต่างเพดานแบบสุมกับ chronological ของ 2017 รว 70% ถึง 99% มาจาก Brute
Force กับ Bot ผลนี้กระทบการอ่าน Recovery % ของทิศ 2018 ไป 2017 และข้อความ "transfer
จริงไม่ได้เปลี่ยน" ในรายงานรอบ 5 หัวข้อ 4.3 (คะแนนข้ามชุดเปลี่ยนด้วย) ยังไม่ได้ปรับ protocol

โค้ดที่ใช้

ที่อยู่	หน้าที่
src/crossdataset/schema.py	mapping 77 features
src/crossdataset/labels.py	mapping 7 คลาสรวม
src/crossdataset/cleaning.py	กฎ clean รวมสองฝั่ง
src/crossdataset/sampling.py	สุ่ม 300k รักษา rare class
src/crossdataset/splits.py	แบ่ง 60/10/30 สองแบบ
src/crossdataset/loaders.py	โหลดสองชุดให้อยู่ในรูปแบบเดียวกัน
scripts/run_crossdataset.py	runner รอบ 4
scripts/analyze_crossdataset.py	สรุปผลเป็นตาราง (summary_gap, summary_leakage, summary_per_class, summary_support และ summary_macro_f1 ที่เพิ่ม macro-F1 $\pm$ SD)
scripts/audit_chronological_split_2017.py	ตรวจว่าลำดับแฉาของ 2017 เรียงตามอะไรและสะท้อนเวลาได้แค่ไหน (ผลอยู่ที่ results/split_audit/cicids2017/)
tests/test_crossdataset_*.py	56 tests

สคริปต์เฉพาะกิจของรอบ 2 และ 3 เก็บไว้ในโฟลเดอร์ code/ ของรอบนั้น ๆ